Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-147
画面名	Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証
会社名	
作成日	

概要

概要説明

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証の入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	入力値・画面状態

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

AnsiTextUtil

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「AnsiTextUtil」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

byteLength

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「byteLength」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

isEncodable

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「isEncodable」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

isWithinBytes

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「isWithinBytes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「条件2を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件3を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件4を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

isValidNotes

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「isValidNotes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

validateNotes

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「validateNotes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「条件4を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 「条件6を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件7を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件8を満たす（原本条件は補足説明を参照）・条件9を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

truncate

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「truncate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 「条件14を満たす（原本条件は補足説明を参照）、またはsource.length() が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理15：詳細仕様参照）。
2. 関連情報を取得・設定する（処理16：詳細仕様参照）。
3. 繰返し条件「条件17を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

5.2. 繰返しの各回：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.3. 繰返しの各回：「条件20を満たす（原本条件は補足説明を参照）、または条件21を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.3.1. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

5.4. 繰返しの各回：関連情報を取得・設定する（処理23：詳細仕様参照）。

5.5. 繰返しの各回：関連情報を取得・設定する（処理24：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件25を満たす（原本条件は補足説明を参照）、またはsource.length() が等しい 0	処理結果を呼出元へ返す
NOT (maxBytes <= 0 || source.length() が等しい 0)	処理結果を呼出元へ返す

toAnsiBytes

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「toAnsiBytes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 関連情報を取得・設定する（処理9：詳細仕様参照）。
4. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

4.2. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

4.3. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

4.4. 通常処理：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

4-例外1. 例外条件「CharacterCodingException e」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

到達条件	結果
条件13を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

fromAnsiBytes

処理概要

Access StrConv(..., vbFromUnicode) 相当の Windows-31J バイト長検証で「fromAnsiBytes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 関連情報を取得・設定する（処理5：詳細仕様参照）。
3. 関連情報を取得・設定する（処理6：詳細仕様参照）。
4. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

4-例外1. 例外条件「CharacterCodingException e」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

到達条件	結果
条件7を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
isWithinBytes	条件1	maxBytes < 0
isWithinBytes	条件2	maxBytes < 0
isWithinBytes	条件3	maxBytes < 0
isWithinBytes	条件4	NOT (maxBytes < 0)
validateNotes	条件1	length > NOTES_MAX_BYTES
validateNotes	処理2	int length = byteLength(value)
validateNotes	条件3	!isEncodable(value)
validateNotes	条件4	!isEncodable(value)

イベント	参照番号	原本条件・処理
validateNotes	処理5	int length = byteLength(value)
validateNotes	条件6	length > NOTES_MAX_BYTES
validateNotes	条件7	!isEncodable(value)
validateNotes	条件8	NOT (!isEncodable(value))
validateNotes	条件9	length > NOTES_MAX_BYTES
truncate	条件1	offset < source.length()
truncate	処理2	offset += Character.charCount(codePoint)
truncate	処理3	result.append(next)
truncate	処理4	break
truncate	条件5	!isEncodable(next)
truncate	条件6	byteLength(result.toString() + next) > maxBytes
truncate	処理7	String next = new String(Character.toChars(codePoint))
truncate	処理8	int codePoint = source.codePointAt(offset)
truncate	処理9	int offset = 0
truncate	処理10	StringBuilder result = new StringBuilder()
truncate	条件11	maxBytes <= 0
truncate	処理12	String source = value == null ? "" : value
truncate	処理13	String source = value == null ? "" : value
truncate	条件14	maxBytes <= 0

イベント	参照番号	原本条件・処理
truncate	処理15	StringBuilder result = new StringBuilder()
truncate	処理16	int offset = 0
truncate	条件17	offset < source.length()
truncate	処理18	int codePoint = source.codePointAt(offset)
truncate	処理19	String next = new String(Character.toChars(codePoint))
truncate	条件20	!isEncodable(next)
truncate	条件21	byteLength(result.toString() + next) > maxBytes
truncate	処理22	break
truncate	処理23	result.append(next)
truncate	処理24	offset += Character.charCount(codePoint)
truncate	条件25	maxBytes <= 0
toAnsiBytes	処理1	buffer.get(bytes)
toAnsiBytes	処理2	byte[] bytes = new byte[buffer.remaining()]
toAnsiBytes	処理3	ByteBuffer buffer = encoder.encode(CharBuffer.wrap(value == null ? "" : value))
toAnsiBytes	処理4	encoder.onUnmappableCharacter(CodingErrorAction.REPORT)
toAnsiBytes	処理5	encoder.onMalformedInput(CodingErrorAction.REPORT)
toAnsiBytes	処理6	CharsetEncoder encoder = WINDOWS_31J.newEncoder()
toAnsiBytes	処理7	CharsetEncoder encoder = WINDOWS_31J.newEncoder()
toAnsiBytes	処理8	encoder.onMalformedInput(CodingErrorAction.REPORT)

イベント	参照番号	原本条件・処理
toAnsiBytes	処理9	encoder.onUnmappableCharacter(CodingErrorAction.REPORT)
toAnsiBytes	処理10	ByteBuffer buffer = encoder.encode(CharBuffer.wrap(value == null ? "" : value))
toAnsiBytes	処理11	byte[] bytes = new byte[buffer.remaining()]
toAnsiBytes	処理12	buffer.get(bytes)
toAnsiBytes	条件13	CharacterCodingException e
fromAnsiBytes	処理1	decoder.onUnmappableCharacter(CodingErrorAction.REPORT)
fromAnsiBytes	処理2	decoder.onMalformedInput(CodingErrorAction.REPORT)
fromAnsiBytes	処理3	CharsetDecoder decoder = WINDOWS_31J.newDecoder()
fromAnsiBytes	処理4	CharsetDecoder decoder = WINDOWS_31J.newDecoder()
fromAnsiBytes	処理5	decoder.onMalformedInput(CodingErrorAction.REPORT)
fromAnsiBytes	処理6	decoder.onUnmappableCharacter(CodingErrorAction.REPORT)
fromAnsiBytes	条件7	CharacterCodingException e